

Лабораторная работа № 1.
ТЕСТИРОВАНИЕ МЕТОДОМ ЧЕРНОГО ЯЩИКА

1. Написать спецификацию для программы по варианту (требования к входным данным, ожидаемые выходные данные, результат в случае ошибки)
2. Написать код для этой программы на любом языке программирования
3. Подготовить тесты по методикам стратегии “черного ящика” – эквивалентное разбиение, АГС, анализ причинно-следственных связей.
4. Предлагаемые тесты свести в таблицу:

Идентификатор теста	Назначение теста	Значение исходных данных	Ожидаемый результат	Реакция программы	Вывод

В отчете должны быть отражены:

- 1.1 Тестовые наборы с использованием метода эквивалентного разбиения
- 1.2 Скриншот с тестами с использованием метода эквивалентного разбиения.
- 1.3 Гипотеза для локализации ошибки, если ошибка была.
- 2.1 Тестовые наборы с использованием метода анализа граничных условий
- 2.2. Скриншот с тестами с использованием метода анализа граничных условий.
- 2.3 При наличии ошибки в программе, сформулировать гипотезу для локализации ошибки.
- 3.1 Тестовые наборы с использованием анализа причинно-следственных связей
- 3.2 Скриншот с тестами с использованием метода анализа причинно-следственных связей.
- 3.3 Гипотеза для локализации ошибки, если ошибка была.
4. Вывод о результативности тестирования с использованием стратегии "черного ящика".

Теоретические сведения

Одним из способов проверки программ является стратегия тестирования методом «**черного ящика**».

Основная идея в тестировании системы как черного ящика состоит в том, что **все материалы, которые доступны тестировщику - это требования на систему**, описывающие ее поведение, и **сама система**, работать с которой он может, только подавая на ее входы некоторые внешние воздействия и наблюдая на выходах некоторый результат.

- Основная задача тестировщика для данного метода тестирования состоит в последовательной проверке соответствия поведения системы требованиям.
- Кроме того, тестировщик должен проверить работу системы в критических ситуациях - что происходит в случае подачи неверных входных значений.
- В идеальной ситуации все варианты критических ситуаций должны быть описаны в требованиях на систему и тестировщику остается только придумывать конкретные проверки этих требований.

Отчеты об обоих типах проблем документируются и передаются разработчикам.

При этом **проблемы первого типа обычно вызывают изменение программного кода**, гораздо реже - изменение требований. Изменение требований в данном случае может потребоваться из-за их противоречивости (несколько разных требований описывают разные модели поведения системы в одной и той же самой ситуации) или некорректности (требования не соответствуют действительности).

Техники тестирования «черным ящиком»

1. Эквивалентное разбиение
2. Анализ граничных значений
3. Анализ причинно-следственных связей (таблицы переходов)

Достоинства метода

- Тестирование методом «черного ящика» позволяет найти ошибки, которые невозможно обнаружить методом «белого ящика».

- «Черный ящик» позволяет быстро выявить ошибки в функциональных спецификациях (в них описаны не только входные значения, но и то, что мы должны в итоге получить).
- Тестировщику не нужна дополнительная квалификация.
- Тестирование проходит «с позиции пользователя».
- Составлять тест-кейсы можно сразу после подготовки спецификации

Недостатки метода

- Возможность пропуска границ и переходов, которые не очевидны из спецификации, но есть в реализации кода.
- Можно протестировать только небольшое количество возможных входных (входящих) значений.
- При отсутствии четкой и полной спецификации проектировать тесты и тест-сценарии оказывается затруднительно.

Метод «черного ящика» удобно применять, если ищем:

- неправильно реализованные функции приложения или сервиса;
- ошибки в пользовательском интерфейсе;
- ошибки в функциональных спецификациях.

Эквивалентное разбиение

Основу метода составляют положения: Исходные данные программы необходимо разбить на конечное число классов эквивалентности. Каждый тест должен включать по возможности максимальное количество различных входных условий, что позволяет минимизировать общее число необходимых тестов. Первое положение используется для разработки набора "интересных" условий, которые должны быть протестированы, а второе - для разработки минимального набора тестов.

Разработка тестов методом эквивалентного разбиения осуществляется в два этапа:

- выделение классов эквивалентности;
- построение тестов.

Выделение классов эквивалентности

Классы эквивалентности выделяются путем выбора каждого входного условия (обычно это предложение или фраза из спецификации) и разбиением его на две или более групп:

Входные условия	Правильные классы эквивалентности	Неправильные классы эквивалентности
Правильные классы – правильные входные данные Неправильные классы – все остальные входные условия Правила выделения классов:		
Входное условие описывает область значений (например, целые числа от 1 до 999)	Правильный класс один: $1 \leq x \leq 999$ x – целое число	Неправильные классы: $x < 1$ $x > 999$
Входное условие описывает число значений (в корзину влезает 7 яблок)	Правильный класс один: Числа от 1 до 7	Неправильные классы: Ни одного яблоко Больше 7 яблок
Если входное условие описывает множество входных значений: На портале могут быть	Правильный класс для каждого значения: - студент - преподаватель	Один неправильный класс: бухгалтер

зарегистрированы студенты, преподаватели, аспиранты	- аспирант	
Если входное условие описывает ситуацию «должно быть» (пароль начинается с заглавной буквы)	Один правильный класс: Первый символ – заглавная буква	Один неправильный класс - Первый символ не заглавная буква

Если есть основание считать, что различные элементы класса эквивалентности трактуются программой неодинаково, то данный класс разбивается на меньшие классы эквивалентности

Пример:

Есть поле ввода с диапазоном допустимых значений от 1 до 100.

Исходя из того, с одной стороны, все допустимые значения могут влиять на поле ввода одинаково, следовательно все числа от 1 до 100 можно смело считать эквивалентными.

С другой стороны, все недопустимые значения должны одинаково влиять на поле ввода (в идеале не должно быть возможности ввода этих значений в поле). Таким образом, есть уже несколько классов эквивалентности:

Допустимые значения (от 1 до 100);

Недопустимые значения:

от $-\infty$ до 0;

от 101 до $+\infty$;

специальные символы (# @ + — / _ : ; “ ‘ и т.д.);
буквы

Построение тестов

Этот шаг заключается в использовании классов эквивалентности для построения тестов. Этот процесс включает в себя:

- Назначение каждому классу эквивалентности уникального номера.
- Проектирование новых тестов, каждый из которых покрывает как можно большее число непокрытых классов эквивалентности, до тех пор, пока все правильные классы не будут покрыты (только не общими) тестами.

- Запись тестов, каждый из которых покрывает один и только один из непокрытых неправильных классов эквивалентности, до тех пор, пока все неправильные классы не будут покрыты тестами.

Недостаток метода эквивалентных разбиения - не исследуется комбинации входных условий.

Анализ граничных значений

Граничные условия - это ситуации, возникающие на, выше или ниже границ входных классов эквивалентности.

Общие правила метода:

1. Проверить граничные значения
2. Проверить значения перед границей
3. Проверить значения после границы

Алгоритм использования:

- выделить классы эквивалентности
- определить граничные значения этих классов
- провести тесты по проверке значения до границы, на границе и сразу после границы.

Количество тестов для проверки граничных значений будет равно количеству границ, умноженному на 3.

Типы границ:

- физическая – физически нельзя преодолеть
- логическая – ограничение, накладываемое логикой (в минуте 60 секунд, возраст не может быть меньше нуля и больше, например 200)
- технологическая – ограничение, накладываемое используемой технологией (поле integer 2 147 483 647)
- произвольная – ограничение, накладываемое аналитиком или разработчиком (в корзину можно положить не более 100 товаров)

Анализ причинно-следственных связей

Метод анализа причинно-следственных связей помогает системно выбирать высокорезультативные тесты. Он дает полезный побочный эффект, позволяя обнаруживать неполноту и неоднозначность исходных спецификаций.

Алгоритм построение подобных тестов:

1. Спецификация разбивается на «рабочие» участки, так как таблицы причинно-следственных связей становятся громоздкими при применении метода к большим спецификациям.
2. В спецификации определяются множество причин и множество следствий. Причина есть отдельное входное условие или класс эквивалентности входных условий. Следствие есть выходное условие или преобразование системы. Каждому причине и следствию приписывается отдельный номер.
3. На основе анализа семантического (смыслового) содержания спецификации строится таблица истинности, в которой последовательно перебираются все возможные комбинации причин и определяются следствия каждой комбинации причин.
4. Каждая строка таблицы истинности преобразуется в тест.
5. Таблица принятия решений, как правило разделяется следующим образом:

Условия	Варианты выполнения действий
Действия	Необходимость действий

Условия – список возможных условий

Варианты – комбинация из выполнения/невыполнения условий из списка

Действия – список возможных действий

Необходимость – указание на то, надо или нет выполнять соответствующее действие для каждой таблицы.

Пример 1:

Работник компании получает бонус, если проработал больше года и достигли согласованных целей.

Таблица расчета бонуса:

		T1	T2	T3	T4	T5	T6	T7	T8
Условия									
Усл1	Работает больше года?	Yes	No	Yes	No	Yes	No	Yes	No
Усл2	Цель согласована?	No	No	Yes	Yes	No	No	Yes	Yes
Усл3	Цель достигнута?	No	No	No	No	Yes	Yes	Yes	Yes
Действие									
	Выплата бонуса?	No	No	No	No	No	No	Yes	No

Тесты T5 и T6 можно удалить – противоречат логике

Пример 2:

Пароль должен содержать:

- 1) Состоять как минимум из 12 символов
- 2) Должен содержать и буквы и цифры
- 3) Пароль не должен совпадать с предыдущим

Условия	Варианты
12 символов	Y, N
Содержит буквы и цифры	Y, N
Пароль не должен совпадать	Y, N

Действия – пароль действительный

Полная таблица действий:

		T1	T2	T3	T4	T5	T6	T7	T8
Условия									
Усл1	12 символов	Yes	Yes	Yes	Yes	No	No	No	No
Усл2	Содержит буквы и цифры	Yes	Yes	No	No	Yes	Yes	No	No
Усл3	не должен совпадать	Yes	No	Yes	No	Yes	No	Yes	No
Действие									
	Пароль действителен	Yes	No	No	No	No	No	No	No

Таблица без избыточных действий:

		T1	T2	T3	T4	T5	T6	T7	T8
Условия									
Усл1	12 символов	Yes	Yes	Yes	Yes	No	No	No	No
Усл2	Содержит буквы и цифры	Yes	Yes	No	No	Yes	Yes	No	No
Усл3	не должен совпадать	Yes	No	-	-	-	-	-	-
Действие									
	Пароль действителен	Yes	No	No	No	No	No	No	No

Таблица без дублирующихся тестов:

		T1	T2	T3	T5	T8
Условия						
Усл1	12 символов	Yes	Yes	Yes	No	No
Усл2	Содержит буквы и цифры	Yes	Yes	No	Yes	No
Усл3	не должен совпадать	Yes	No	-	-	-
Действие						
	Пароль действителен	Yes	No	No	No	No

Общая стратегия тестирования

Все методологии проектирования тестов могут быть объединены в общую стратегию. Это оправдано тем, что каждый метод обеспечивает создание определенного набора тестов, но ни один из них сам по себе не может дать полный набор тестов. Приемлемая стратегия состоит в следующем:

1. Использовать анализ граничных значений.
2. Определить правильные и неправильные классы эквивалентности для входных и выходных данных и дополнить, если это необходимо, тесты, построенные на предыдущих шагах.
3. Для получения дополнительных тестов рекомендуется использовать метод предположения об ошибке.